

Relazione - Versione 4: Introduzione a TensorFlow

Introduzione

La Versione 4 ricostruisce con TensorFlow il modello linguistico neurale a caratteri della Versione 3.

Corpus, vocabolario, compito basato su caratteri adiacenti, rappresentazione numerica, matrice dei pesi 27 per 27, suddivisione training/validation, mini-batch mescolati, obiettivo cross-entropy e generatore autoregressivo restano invariati. I tensori e le variabili TensorFlow sostituiscono gli array NumPy nei calcoli del modello, mentre la differenziazione automatica sostituisce i gradienti manuali.

```
Caratteri
↓
Identificatori interi
↓
Tensori one-hot TensorFlow
↓
tf.Variable addestrabile
↓
Gradienti automatici
↓
Aggiornamenti dei pesi
↓
Testo generato
```

Obiettivo della Versione 4

Il modello deve essere in grado di:

- riutilizzare corpus, vocabolario, codifica numerica e architettura neural bigram della Versione 3;
- rappresentare gli input one-hot come tensori TensorFlow e i pesi come `tf.Variable`;
- calcolare una cross-entropy stabile direttamente dai logits con TensorFlow;
- calcolare automaticamente i gradienti dei pesi con `tf.GradientTape`;
- aggiornare i pesi con la stessa procedura mini-batch gradient descent;
- conservare il monitoraggio training/validation e la generazione riproducibile;
- verificare che i gradienti automatici coincidano con la formula manuale della Versione 3.

Architettura

Il flusso completo della Versione 4 è:

```
Corpus: 440 caratteri
↓
Vocabolario: 27 caratteri
↓
439 coppie input-target adiacenti
↓
Tensore one-hot TensorFlow: (439, 27)
↓
tf.Variable addestrabile: (27, 27)
↓
Split riproducibile 80 / 20
↓
Aggiornamenti mini-batch con GradientTape
↓
Monitoraggio training e validation loss
↓
Generatore autoregressivo
```

Il contesto contiene ancora un solo carattere. La Versione 4 resta quindi lo stesso modello neural bigram a caratteri usato nella Versione 3.

Split dei dati e mini-batch invariati

Un generatore NumPy locale con seed 42 crea la stessa permutazione riproducibile dei 439 indici. Le operazioni gather di TensorFlow selezionano i tensori di training e validation corrispondenti.

```
esempi di training = 351
esempi di validation = 88
batch size = 32
aggiornamenti per epoca = 11
```

Soltanto gli esempi di training vengono mescolati e usati per aggiornare i parametri. Gli esempi di validation misurano la loss senza modificare il modello. Lo split a livello di esempio conserva le limitazioni documentate nella Versione 3.

Modello numerico TensorFlow

Ogni carattere di input è rappresentato da un tensore one-hot TensorFlow di lunghezza 27. Il modello conserva la matrice dei pesi addestrabile 27 per 27 in una `tf.Variable`.

```
Forma tensore di input: (439, 27)
Forma matrice dei pesi: (27, 27)
Parametri addestrabili: 729
Tipo dei tensori: float32
```

Un generatore casuale TensorFlow inizializzato con seed 42 produce i pesi iniziali. La variabile mantiene una forma fissa e può essere aggiornata direttamente durante il training.

Loss TensorFlow stabile

La moltiplicazione matriciale TensorFlow produce i logits. La sparse softmax cross-entropy misura quindi l'errore direttamente dai logits e dagli ID target interi.

```
logits = tf.matmul(inputs, weights)
example_losses = tf.nn.sparse_softmax_cross_entropy_with_logits(
    labels=targets, logits=logits
)
loss = tf.reduce_mean(example_losses)
```

L'operazione combinata di TensorFlow gestisce internamente la stabilità numerica. Softmax viene ancora usata separatamente quando servono probabilità per ispezione e generazione.

Esecuzione controllata e riproducibilità

Il notebook seleziona esplicitamente l'esecuzione CPU perché questo piccolo modello non richiede CUDA. I log del backend TensorFlow vengono soppressi per mantenere pulito l'output.

Seed espliciti controllano l'inizializzazione dei pesi TensorFlow, split NumPy, shuffling dei mini-batch e campionamento del testo. Le riesecuzioni sono deterministiche nello stesso ambiente software.

Differenziazione automatica

Per ogni mini-batch, TensorFlow registra i calcoli forward e deriva automaticamente la loss rispetto alla matrice dei pesi addestrabile:

- `tf.GradientTape` registra le operazioni usate per calcolare la loss;
- `tape.gradient` calcola automaticamente il gradiente dei pesi;
- `assign_sub` applica lo stesso aggiornamento gradient descent della Versione 3;
- training e validation loss vengono misurate dopo ogni epoca.

```
with tf.GradientTape() as tape:
    batch_loss = calculate_loss(weights, inputs, targets)
    weights_gradient = tape.gradient(batch_loss, weights)
    weights.assign_sub(learning_rate * weights_gradient)
```

Configurazione del training:

```
Learning rate: 1.0
Batch size: 32
Epoche: 100
Seed: 42
```

Risultato del training

```
Training loss iniziale: 3.295990
Training loss finale: 1.987506
Validation loss iniziale: 3.297369
Validation loss finale: 2.903532
Validation loss migliore: 2.893411
Epoca migliore: 64
```

La training loss diminuisce per tutta l'esecuzione. La validation loss inizialmente diminuisce, raggiunge il minimo all'epoca 64 e poi aumenta leggermente, perché soltanto la training loss viene ottimizzata direttamente.

I risultati restano molto vicini alla Versione 3. Le piccole differenze derivano dal generatore casuale TensorFlow e dai calcoli float32. Il notebook registra l'epoca migliore per l'analisi ma usa i pesi finali dell'epoca 100 per generare testo.

Verifica dei gradienti

I test confrontano il gradiente automatico TensorFlow con la formula manuale introdotta nella Versione 3.

```
automatic_gradient = tape.gradient(loss, weights)
target_vectors = tf.one_hot(targets, depth=vocabulary_size)
logits_gradient = (probabilities - target_vectors) / batch_size
manual_gradient = inputs.T @ logits_gradient
np.allclose(auto_gradient, manual_gradient)
```

La corrispondenza mostra che la differenziazione automatica esegue lo stesso calcolo fondamentale implementato manualmente nella versione precedente.

Probabilità apprese

Dopo il training, una riga della variabile dei pesi TensorFlow può ancora essere interpretata come distribuzione del carattere successivo per un dato contesto.

Per il carattere m, le probabilità più alte sono:

```
o: 0.2497 e: 0.1818 a: 0.1810
s: 0.1126 p: 0.1121 spazio: 0.0428
```

Generazione neurale del testo

TensorFlow calcola la distribuzione di probabilità, mentre un generatore NumPy locale campiona il carattere successivo. Il carattere campionato diventa il nuovo input, conservando il ciclo autoregressivo della Versione 3.

La dimostrazione genera 300 caratteri dai pesi finali. Lo stesso seed riproduce lo stesso testo generato.

Controlli inclusi

Le asserzioni verificano tipi di tensori e variabili TensorFlow, forme one-hot e dei parametri, split dei dati, loss finite, probabilità normalizzate, corrispondenza tra gradienti manuali e automatici, ripetibilità del training, generazione riproducibile e lunghezza richiesta.

Un'esecuzione completa termina con il messaggio All checks passed.

Cosa insegna la Versione 4

La progressione centrale è:

tensor -> variable -> loss -> tape -> gradient -> update -> verify -> sample

La Versione 4 collega il training numerico manuale al training basato su framework. Il modello predittivo resta volutamente invariato, così che l'attenzione didattica rimanga su tensori, variabili e differenziazione automatica TensorFlow.

Interpretazione

TensorFlow non rende il modello più espressivo. Automatizza la differenziazione e fornisce operazioni stabili sui tensori, conservando la stessa architettura bigram a caratteri e la stessa procedura gradient descent.

La stretta corrispondenza con la Versione 3, insieme al confronto esplicito dei gradienti, mostra che entrambe le implementazioni ottimizzano lo stesso obiettivo.

Limitazioni

- il contesto contiene un solo carattere;
 - il corpus è piccolo e incorporato nel notebook;
 - lo split casuale a livello di esempio può collocare transizioni ripetute sia nel training sia nella validation;
 - l'esecuzione è eager e basata su CPU, senza una pipeline di input ottimizzata;
 - non sono presenti early stopping, ripristino di checkpoint, hidden layer o embedding;
 - la validation non è una stima robusta della generalizzazione a un corpus differente;
 - il testo generato conserva regolarità locali ma non coerenza a lungo termine.
-

Conclusione

La Versione 4 ricostruisce con TensorFlow il modello neurale a caratteri NumPy, conservando dati, architettura, training e generatore. Tensori e variabili TensorFlow rappresentano il modello, la differenziazione automatica sostituisce i gradienti manuali e la verifica diretta collega le due implementazioni.

Il risultato prepara il progetto alla Versione 5, dove embedding appresi e finestre di contesto più ampie potranno aumentare la capacità rappresentativa del modello.